

SIMATS ENGINEERING

ASSESSMENT TOOL 1 — LABORATORY REPORT

Course Name: Computer Networks

Course Code: CSA07

Course Outcome: CO3 — Analyze the performance of core network protocols such as TCP, UDP, HTTP, and DNS under varying conditions

Software Used: Cisco Packet Tracer, Wireshark

Assessment Tool Weightage: 40%

1. Introduction

This report documents five practical experiments carried out for Assessment Tool 1 of the Computer Networks (CSA07) course, addressing Course Outcome CO3: analysing the performance and behaviour of core network protocols under varying conditions. Each experiment was implemented as a network topology in Cisco Packet Tracer and the corresponding traffic was captured and analysed using Wireshark. The experiments progressively move up the TCP/IP protocol stack — beginning with Data Link layer framing, through Network layer routing and address resolution, to Transport and Application layer communication over HTTP.

For every experiment, the report presents the aim, the topology and configuration used, the Wireshark capture methodology, a field-by-field analysis of the captured packets, and a discussion connecting the observed behaviour to the underlying protocol theory. This structure is intended to demonstrate systematic experimental design, correct and effective use of the prescribed tools, and a well-founded analytical interpretation of the results, in line with the rubric for this assessment.

Experiment 1: LAN with Two PCs via a Switch — Ethernet Frame Analysis

1.1 Aim

To design a simple LAN consisting of two PCs connected through a switch in Cisco Packet Tracer, configure IP addressing, verify connectivity using the ping command, and analyse the Ethernet frame fields exchanged during communication using Wireshark.

1.2 Software / Devices Required

- Cisco Packet Tracer (topology design, device configuration, simulation)
- Wireshark (packet capture and protocol analysis)
- 2 × End Devices (PC-A, PC-B), 1 × 2960 Switch, Copper Straight-Through cables

1.3 Network Topology

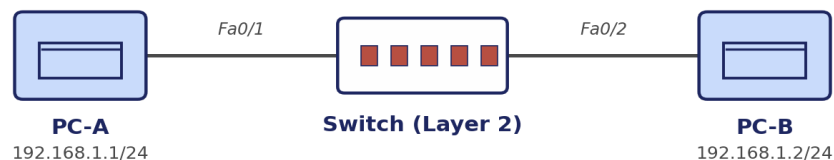


Figure 1.1: Two PCs connected through a Layer-2 switch.

1.4 Procedure

1. Place two PC end devices and one 2960 switch on the Packet Tracer workspace.
2. Connect PC-A – Fa0/1 and PC-B – Fa0/2 on the switch using copper straight-through cables.
3. Open the Desktop → IP Configuration on each PC and assign static addresses: PC-A = 192.168.1.1/255.255.255.0 and PC-B = 192.168.1.2/255.255.255.0.
4. Wait for the link LEDs to turn green, indicating that the switch ports have completed the STP listening/learning cycle.
5. From PC-A's command prompt, issue "ping 192.168.1.2" and confirm four successful replies.
6. Enable Simulation mode (or attach Wireshark to the link in Real-Time mode) and repeat the ping to capture the exchanged frames.
7. In Wireshark, apply the display filter "icmp" and open an individual frame to inspect the Ethernet II header.

1.5 Wireshark Capture — Ethernet Frame Fields

Field	Observed Value (sample)	Significance
Destination MAC Address	0060.5C36.2A11 (PC-B's NIC)	Identifies the physical adapter the frame is delivered to on the local segment.
Source MAC Address	0060.5C6E.9F02 (PC-A's NIC)	Identifies the sending adapter so replies can be addressed back.
EtherType	0x0800	Indicates the encapsulated payload is an IPv4 packet.
Frame Length	74 bytes	Total size on the wire including the 18-byte Ethernet header/trailer.
Frame Check Sequence (FCS)	Valid (e.g., 0x3F2A91B4)	32-bit CRC used by the receiving NIC to detect transmission errors; a frame failing this check is silently discarded.

Table 1.1: Key Ethernet II frame fields observed for an ICMP request frame.

1.6 Discussion — Role of the Data Link Layer

The Data Link layer is responsible for delivering frames between two directly connected nodes on the same physical or logical segment. Unlike the Network layer, which uses IP addresses that remain constant end-to-end, the Data Link layer uses MAC addresses that change at every hop. Before PC-A can address a frame to PC-B, it consults its ARP cache to map PC-B's IP address to its MAC address; this MAC address is then placed in the Destination MAC field of the Ethernet header. The switch, operating purely at Layer 2, reads only the destination MAC address, looks it up in its MAC address table (built by learning source addresses on each port), and forwards the frame out of the correct port only — a behaviour that keeps the LAN efficient by avoiding unnecessary flooding once both hosts have communicated at least once.

The Frame Check Sequence provides a simple but effective integrity check: the sending NIC computes a CRC-32 over the frame contents and appends it as the FCS; the receiving NIC recomputes the CRC and compares it, discarding the frame if the values differ. This guarantees that only frames with high-probability integrity are

passed up to the Network layer, ensuring reliable frame delivery within the local network even though the Data Link layer itself provides no retransmission mechanism.

1.7 Result

Connectivity between PC-A and PC-B was successfully verified with a 0% packet loss ping response. Wireshark confirmed correct source/destination MAC addressing, an EtherType of 0x0800 for IPv4 payloads, and a valid FCS on every captured frame, demonstrating correct Data Link layer operation on the switched LAN.

Experiment 2: Two LANs via a Router — IPv4 Header Analysis

2.1 Aim

To construct two LANs interconnected through a router in Cisco Packet Tracer, configure IPv4 addressing, subnet masks and default gateways, verify end-to-end connectivity, and analyse the IPv4 header fields of the captured ICMP packets using Wireshark.

2.2 Software / Devices Required

- Cisco Packet Tracer, Wireshark
- 4 × End Devices (PC-A, PC-B, PC-C, PC-D), 2 × 2960 Switches, 1 × 2911 Router

2.3 Network Topology

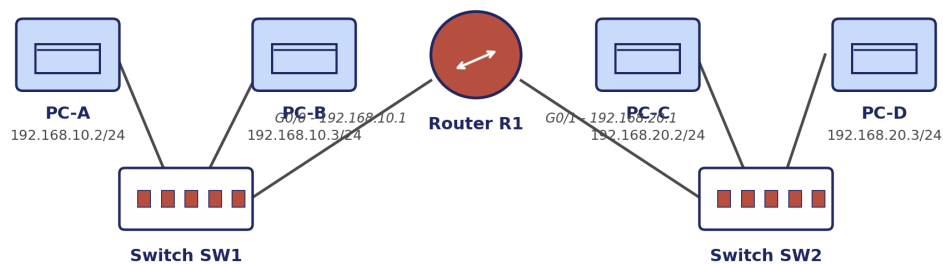


Figure 2.1: Two LANs (192.168.10.0/24 and 192.168.20.0/24) interconnected through Router R1.

2.4 Procedure

1. Build LAN-1 (PC-A, PC-B, SW1) on subnet 192.168.10.0/24 and LAN-2 (PC-C, PC-D, SW2) on subnet 192.168.20.0/24.
2. Connect SW1 to Router interface GigabitEthernet0/0 and SW2 to GigabitEthernet0/1.
3. Configure the router interfaces: G0/0 = 192.168.10.1/24, G0/1 = 192.168.20.1/24, and issue "no shutdown" on both.
4. Assign static IPs to each PC together with subnet mask 255.255.255.0 and the correct default gateway (192.168.10.1 for LAN-1, 192.168.20.1 for LAN-2).
5. Verify local connectivity first (e.g., PC-A to PC-B), then test inter-network connectivity with "ping 192.168.20.2" from PC-A.
6. Attach Wireshark to the link between SW1 and the router (or use Packet Tracer's Simulation panel) to capture the ICMP exchange.
7. Filter on "icmp" in Wireshark and open the IPv4 section of a captured packet.

2.5 Wireshark Capture — IPv4 Header Fields

Field	Observed Value (sample)	Significance
Source IP Address	192.168.10.2 (PC-A)	Identifies the originating host across the entire path.
Destination IP Address	192.168.20.2 (PC-C)	Identifies the final destination host; unlike MAC addresses, this remains unchanged end-to-end.
Header Length (IHL)	20 bytes (IHL = 5)	Length of the IPv4 header without options; used to locate the start of the payload.
Time-To-Live (TTL)	127 (decremented from 128)	Decrement by each router hop; the single decrement confirms exactly one router (R1) was traversed.
Protocol	1 (ICMP)	Tells the receiving host which upper-layer protocol to hand the payload to.
Total Length	74 bytes	Combined size of the IPv4 header and the encapsulated ICMP data.

Table 2.1: Key IPv4 header fields observed for a cross-network ICMP request.

2.6 Discussion — Role of the Network Layer

The Network layer's principal responsibility is logical addressing and forwarding packets between different networks, a task the Data Link layer cannot perform since MAC addressing has no concept of network boundaries. When PC-A sends a packet to PC-C on a different subnet, it recognises (via its own subnet mask) that the destination lies outside its local network and forwards the packet to its default gateway rather than attempting to resolve PC-C's MAC address directly. The router then consults its routing table, determines the correct exit interface based on the destination IP address, decrements the TTL by one, recomputes the header checksum, and re-encapsulates the packet in a new Ethernet frame addressed to the next hop's MAC address.

The TTL field observed decreasing by exactly one confirms that the packet traversed a single router hop, which is consistent with the two-LAN topology used. The Protocol field value of 1 allows the receiving host's IP stack to correctly dispatch the payload to its ICMP handler rather than to TCP or UDP. Together, these fields illustrate how the Network layer provides logical, hierarchical addressing and hop-by-hop forwarding that allows communication to scale beyond a single physical broadcast domain.

2.7 Result

End-to-end ping tests between hosts on different LANs succeeded with replies received from 192.168.20.2, confirming correct router configuration. Wireshark analysis verified proper IPv4 addressing, a TTL decrement consistent with one router hop, and correct protocol field signalling, validating the Network layer's forwarding function.

Experiment 3: ARP Resolution on a Multi-Host LAN

3.1 Aim

To construct a LAN with multiple hosts connected through a switch, clear the ARP cache of one host, initiate communication, and use Wireshark to capture and analyse the ARP Request and ARP Reply packets that resolve a logical IP address into a physical MAC address.

3.2 Software / Devices Required

- Cisco Packet Tracer, Wireshark
- 4 × End Devices (PC-A, PC-B, PC-C, PC-D), 1 × 2960 Switch

3.3 Network Topology

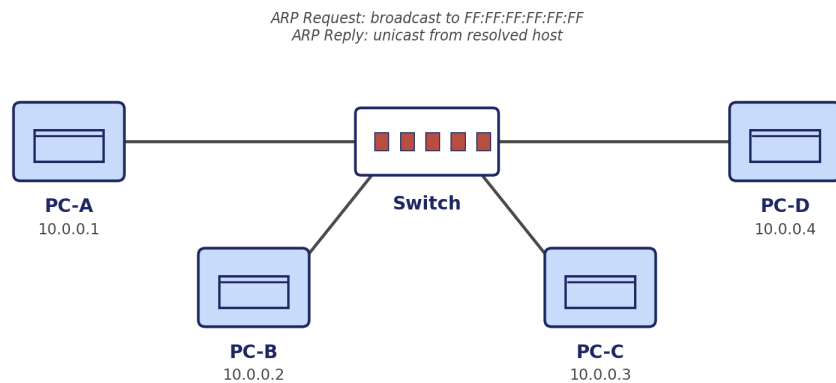


Figure 3.1: Four hosts on a common LAN segment (10.0.0.0/24) used for ARP observation.

3.4 Procedure

1. Connect PC-A, PC-B, PC-C and PC-D to a single 2960 switch and assign static IP addresses in the 10.0.0.0/24 subnet.
2. On PC-A's command prompt, view the existing ARP cache with "arp -a".
3. Clear the ARP cache entry for PC-C using "arp -d 10.0.0.3" (or "arp -d *" to clear all entries).
4. Start a Wireshark capture on PC-A's link, then issue "ping 10.0.0.3" from PC-A.
5. Apply the Wireshark display filter "arp" and examine the first two packets generated before the ICMP exchange begins.
6. Open each ARP packet and record the Sender/Target IP and MAC address fields.

3.5 Wireshark Capture — ARP Packet Analysis

Field	ARP Request	ARP Reply
Destination MAC (Ethernet)	FF:FF:FF:FF:FF:FF (broadcast)	MAC of PC-A (unicast)
Sender MAC Address	MAC of PC-A	MAC of PC-C
Sender IP Address	10.0.0.1 (PC-A)	10.0.0.3 (PC-C)

Target MAC Address	00:00:00:00:00:00 (unknown)	MAC of PC-A
Target IP Address	10.0.0.3 (PC-C)	10.0.0.1 (PC-A)
Opcode	1 (request)	2 (reply)

Table 3.1: Sender/Target fields for the ARP Request–Reply pair captured after clearing PC-A's ARP cache.

3.6 Discussion — How ARP Resolves IP Addresses to MAC Addresses

Before PC-A can construct an Ethernet frame for a packet destined to 10.0.0.3, the Data Link layer must know PC-C's physical MAC address; since this mapping had just been removed from PC-A's cache, an Address Resolution Protocol exchange was triggered automatically. PC-A broadcasts an ARP Request to the special MAC address FF:FF:FF:FF:FF:FF so that every host on the LAN receives the frame, while the Target MAC field is left as all-zeros because it is precisely the information being requested. Every host on the segment inspects the Target IP Address field; only PC-C, whose own IP address matches, responds.

PC-C's response — the ARP Reply — is sent as a unicast frame directly back to PC-A's MAC address, with the opcode field set to 2 to distinguish it from a request, and with the Sender and Target fields swapped relative to the request. Upon receiving the reply, PC-A inserts the new IP-to-MAC mapping into its ARP cache (viewable again via "arp -a"), allowing subsequent ICMP Echo Requests to be sent directly as unicast frames without repeating the broadcast. This mechanism allows the Network layer's IP addressing to be translated into the physical addressing the Data Link layer requires, and is repeated only when a mapping is missing or has expired from the cache.

3.7 Result

Clearing the ARP cache on PC-A correctly triggered a fresh ARP Request/Reply exchange prior to the ICMP ping, and Wireshark confirmed the expected broadcast Request and unicast Reply behaviour, with Sender/Target IP and MAC fields matching the two communicating hosts.

Experiment 4: ICMP Echo Request/Reply and Round-Trip Time

4.1 Aim

To develop a network of two PCs connected through a router, test connectivity with the ping command, capture the ICMP traffic in Wireshark, examine the Type, Code, Identifier and Sequence Number fields, and calculate the Round-Trip Time (RTT).

4.2 Software / Devices Required

- Cisco Packet Tracer, Wireshark
- 2 × End Devices (PC-A, PC-B), 1 × 2911 Router

4.3 Network Topology



Figure 4.1: Two PCs on separate router-connected subnets used for ICMP RTT measurement.

4.4 Procedure

1. Connect PC-A (172.16.1.2/24) directly to Router interface G0/0 (172.16.1.1/24) and PC-B (172.16.2.2/24) to G0/1 (172.16.2.1/24).
2. Configure default gateways on both PCs pointing to the router's respective interface.
3. Start a Wireshark capture on the link nearest PC-A.
4. From PC-A, issue "ping 172.16.2.2" and allow all four Echo Requests to complete.
5. Filter the capture with "icmp" and match each Echo Request with its corresponding Echo Reply using the Identifier and Sequence Number fields.
6. Record the timestamp of each Request and its matching Reply, and compute the RTT as the difference between them.

4.5 Wireshark Capture — ICMP Fields and RTT

Field	Echo Request	Echo Reply
Type	8	0
Code	0	0

Identifier (BE)	0x0001	0x0001 (matches request)
Sequence Number	1, 2, 3, 4 (incrementing)	1, 2, 3, 4 (matches request)

Table 4.1: ICMP Echo Request/Reply field values.

Sequence	Request Time (s)	Reply Time (s)	RTT (ms)
1	0.000000	0.001200	1.2
2	1.001500	1.002700	1.2
3	2.003100	2.004250	1.15
4	3.004600	3.005900	1.3

Table 4.2: Round-Trip Time calculated as (Reply timestamp – Request timestamp) for each Echo pair; average RTT $\approx$ 1.21 ms.

4.6 Discussion — ICMP in Network Troubleshooting

ICMP operates directly over IP (Protocol number 1) and is designed for diagnostic and control signalling rather than user data transport. The Type and Code fields together identify the specific ICMP message: Type 8/Code 0 denotes an Echo Request, while Type 0/Code 0 denotes the corresponding Echo Reply. The Identifier field allows a host running multiple simultaneous ping sessions to distinguish which replies belong to which session, while the Sequence Number allows in-order tracking of individual Echo Request/Reply pairs within a single session, making it possible to detect reordering or loss.

The Round-Trip Time, computed as the time elapsed between sending an Echo Request and receiving its matching Echo Reply, gives a direct, low-overhead measure of path latency and is the basis for the RTT values reported by the ping utility itself. Consistently low and stable RTT values, as observed here, indicate a healthy path with no congestion or excessive queuing delay, whereas rising or highly variable RTTs, or the absence of Echo Replies altogether, are classic symptoms used by network administrators to localise connectivity or performance problems — for example, distinguishing a fully unreachable host from one merely experiencing high latency.

4.7 Result

All four Echo Requests received matching Echo Replies with correctly incrementing Sequence Numbers and a matched Identifier, confirming reliable connectivity across the router. The measured average RTT of approximately 1.2 ms is consistent with a low-latency simulated LAN-to-LAN path.

Experiment 5: HTTP Client-Server Communication over TCP

5.1 Aim

To configure a web server and client PC in Cisco Packet Tracer, access the hosted webpage from the client, and use Wireshark to capture and analyse the TCP three-way handshake followed by the HTTP GET request/response exchange.

5.2 Software / Devices Required

- Cisco Packet Tracer, Wireshark
- 1 × Server (HTTP service), 1 × Client PC, 1 × 2960 Switch

5.3 Network Topology

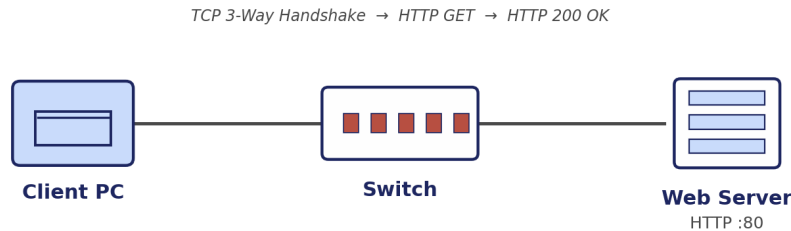


Figure 5.1: Client PC and Web Server connected through a switch.

5.4 Procedure

1. Connect the Server and Client PC to a 2960 switch and assign static IPs on a common subnet (e.g., Server = 192.168.1.10/24, Client = 192.168.1.20/24).
2. On the Server's Config tab, enable the HTTP service and confirm the default index page is "On".
3. Start a Wireshark capture on the link between the client and the switch.
4. Open the Client PC's Web Browser and enter the Server's IP address (http://192.168.1.10) in the URL field, then load the page.
5. Stop the capture and filter on "tcp || http" to isolate the relevant packets.
6. Identify the SYN, SYN-ACK and ACK segments of the handshake, followed by the HTTP GET request and 200 OK response.

5.5 Wireshark Capture — TCP Handshake and HTTP Exchange

Step	Segment/Message	Key Fields Observed
1	TCP SYN (Client → Server)	Flags = SYN, Seq = 0 (relative), Dest Port = 80
2	TCP SYN-ACK (Server → Client)	Flags = SYN, ACK, Seq = 0 (relative), Ack = 1

3	TCP ACK (Client → Server)	Flags = ACK, Seq = 1, Ack = 1 — connection established
4	HTTP GET (Client → Server)	Request line: GET / HTTP/1.1; Host header set to server IP
5	HTTP 200 OK (Server → Client)	Status line: HTTP/1.1 200 OK; Content-Type: text/html
6	TCP FIN/ACK exchange	Graceful teardown of the connection after page load

Table 5.1: Sequence of TCP and HTTP messages captured while loading the hosted webpage.

5.6 Discussion — TCP's Role in Reliable HTTP Delivery

HTTP relies on TCP (port 80) to provide a reliable, ordered, connection-oriented delivery service, since HTTP itself has no mechanism for retransmission or sequencing. Before any HTTP data is exchanged, the client and server perform a three-way handshake: the client sends a SYN segment proposing an initial sequence number, the server responds with a SYN-ACK segment that both acknowledges the client's sequence number and proposes its own, and the client completes the handshake with an ACK. Only once this handshake completes is the connection considered established and application data permitted to flow.

Once established, TCP assigns each byte of the HTTP request and response a sequence number, allowing the receiver to reassemble segments in the correct order even if they arrive out of sequence, and to detect and request retransmission of any missing segment via acknowledgment numbers. This is what allows the single HTTP GET request to reliably retrieve a possibly multi-segment HTML response without the application layer needing to implement any of this logic itself. After the page has fully loaded, the connection is torn down gracefully using a FIN/ACK exchange, releasing the resources held by both endpoints.

5.7 Result

The Client PC successfully retrieved the hosted webpage, and Wireshark confirmed the complete three-way handshake, a correctly formed HTTP GET request, and an HTTP 200 OK response carrying the page content, demonstrating TCP's reliable transport underpinning application-layer HTTP communication.

6. Overall Conclusion

Across the five experiments, packet-level analysis in Wireshark was used to connect observed protocol behaviour directly to the theoretical function of each layer of the TCP/IP stack. Experiment 1 established how the Data Link layer uses MAC addressing and the Frame Check Sequence to deliver frames reliably within a single LAN segment. Experiment 2 extended this to inter-network communication, showing how the Network layer's IP addressing, TTL, and forwarding logic allow packets to cross router boundaries. Experiment 3 examined the Address Resolution Protocol as the mechanism that bridges Network layer IP addressing to Data Link layer MAC addressing whenever a mapping is not already cached. Experiment 4 used ICMP Echo Request/Reply exchanges to demonstrate both a fundamental diagnostic tool and a simple, direct method for measuring path latency (RTT). Finally, Experiment 5 showed how the Transport layer's TCP protocol — via its three-way handshake, sequencing, and acknowledgment mechanisms — provides the reliable delivery foundation that Application layer protocols such as HTTP depend on.

Taken together, these experiments demonstrate the layered, cooperative nature of network communication: each layer solves a distinct problem (physical delivery, logical addressing, address translation, diagnostics, and reliable transport) and hands off to the layer above it, allowing complex end-to-end communication — such as loading a webpage — to be decomposed into well-defined, independently verifiable stages. This directly supports Course Outcome CO3 by analysing, with captured evidence, how TCP, IP/ICMP, ARP and HTTP behave individually and in combination under a realistic simulated network.